CHEW Restaurant Whitepaper

November 2025

Blockchain Cryptocurrency Fall 2025 - Professor Radnay
Victor Chen
Valerie Wu
Yuyao Wang

Contents

1 Abstract

CHEW is an "on-chain shareholder system" that we design for a future restaurant. In simple words, we turn the restaurant's profit rights and major decision rights into a token called **CHEW** and put it on a blockchain. Investors can hold CHEW like shares of the restaurant. All money flows and dividend rules are written into smart contracts, so they are public, transparent, and cannot be changed casually.

The project starts from fundraising. Anyone can use the stablecoin **mUSDT** to buy CHEW in the *Crowdsale* contract. Our goal is to raise 300,000 mUSDT. This amount represents 75% of the restaurant's economic and governance rights. The remaining 25% is kept by the team and locked for 24 months (no transfer or selling during the lockup). This shows that the operators are tied to investors for the long term. After the sale ends, CHEW can trade freely on secondary markets. Holders may receive dividends, but can also gain or lose from price changes.

After the restaurant actually starts to operate, all income and expenses are recorded on-chain by the *RestaurantAccounts* contract. We define one fixed "record date" every year: 31 December. On that day the contract takes a snapshot of all CHEW balances. On 1 January of the next year, it sends out the previous year's distributable profit in mUSDT, according to this snapshot. If a holder does not claim the dividend in time, the money does not expire. It stays in the contract as a claimable balance and can be taken later at any time.

For governance, we give location choice, renovation plans and the annual budget to a *Governor + Timelock* contract. Token holders can create proposals and vote. When a proposal passes, it is not executed immediately. It waits for a delay window and then is executed automatically by the contract. This reduces the risk of "one person makes a quick decision". Lighter community actions, such as menu updates or Top-N dishes each season, are handled by a separate *MenuVoting* contract. This increases community participation, but does not touch the main treasury directly.

2 Introduction

Most physical restaurants still run their equity and governance in a very traditional and opaque way. After money goes into the company bank account, normal investors usually only see a quarterly or annual report. It is hard for them to know how much the restaurant really earns every day, and where the money actually goes. Revenue, cost, rent and salaries are recorded in different systems, at different times, with different definitions. Tracking one unit of sales all the way to "how much distributable profit is left" is almost impossible.

Dividends are also handled in a black-box way. Management calculates them off-chain, decides when to pay, how much to pay, and based on which accounting rule. Outside investors cannot really verify whether the dividend matches the real cash flow. At the same time, the relationship between customers and brands is usually very short-term: you come, eat once, and leave. Almost no one can participate in the long-term value or key decisions of a single restaurant.

Web3 gives us a toolkit that fits this problem quite well. Smart contracts can fix the rules in code. Event logs and on-chain transaction history allow every action to be replayed and audited. In other words, we can map the restaurant's real-world operations into a set of on-chain records: who did what, at what time, with what permission, and what effect it had on balances and rights. Anyone can verify this independently.

CHEW aims to take the profit rights and major decision rights of a restaurant and break them down into reusable on-chain modules. Every revenue entry, every budget approval, every dividend round, and every important vote should have a clear on-chain record and an automatic execution path, instead of living in Excel sheets and WeChat chats.

We also want to clarify a common misunderstanding: CHEW does not mean "every customer who eats here becomes a shareholder". Our starting point is a public fundraising. People who are willing to take economic risk and also join governance can buy CHEW with mUSDT in the Crowdsale contract. Only addresses that actually put in money and hold CHEW will receive profit-sharing and voting power.

After the fundraising ends, the token can trade freely on secondary markets. However, who really gets dividends depends on the "record date" snapshot. We set December 31 as the record date each year. On that day we record how many CHEW each address holds, and on January 1 we distribute the previous year's distributable profit to wallets according to this snapshot. The 25% CHEW held

by the team is locked for two years. During this period, the team cannot sell or transfer these tokens. This design connects team incentives with long-term performance and reduces the chance of "dumping and running away".

To make this idea concrete, we break the restaurant's business logic into five parts:

- **Fundraising** – handled by the Crowdsale contract. It swaps mUSDT for CHEW and builds an on-chain list of shareholders.
- **Equity mapping** – implemented by an ERC20 token with voting features. It records who owns how many tokens, and it also locks the team's 25% for 24 months.
- **On-chain accounting** – handled by the RestaurantAccounts contract. It records income and expenses and calculates the distributable profit for each period.
- **Dividend mechanism** – based on the annual snapshot. It converts profits into mUSDT and assigns them to holders by proportion. If someone does not claim in that year, they can still claim later.
- **Governance and participation** – major decisions (like location, renovation, and big budgets) are decided through Governor + Timelock proposals and votes. Lighter decisions (such as new menu items or Top-N dishes) go through MenuVoting, so the community can co-build the brand in a low-risk way.

In the following sections, we first explain the real-world problems and our assumptions. Then we describe the system architecture, the flow of funds and dividends, the tokenomics, and the governance design in an intuitive way. Finally, we show how we ran the whole pipeline on a testnet, and we summarize the challenges we met and potential next steps.

Our goal is not to design a perfect business model, but to show that with a relatively simple set of smart contracts, it is possible to turn a restaurant's "making money and making decisions" into a transparent and verifiable rule system.

3 Vision & Problem

3.1 Problems in the real world

When we first talked about this project, we started from a very simple question:

"Is this restaurant actually making money? Where does the money go? And why is the dividend like this? As outsiders, we almost never know."

Traditional restaurants have several common pain points:

- **Accounting is not transparent.**
  Revenue sits in the POS system, costs are in Excel, salaries are in another system. In the end you only see one merged report. It is very hard for investors to link "how much we sold today" to "how much I finally received as profit."
- **Dividend rules are unclear.**
  When to distribute, how much to distribute, and based on which accounting standard is usually decided by the owner or the finance team. Investors can only passively accept the result.
- **Decision-making is a black box.**
  Important decisions like location, renovation, and budget are often made by a small group of people. There is almost no traceable record of the process, so it is hard to review or question later.
- **Weak community participation.**
  Customers can only come, eat, and leave. It is hard for them to take part in the long-term building of the brand. Investors also lack a clear and positive way to interact with daily operations of the restaurant.

3.2 Our vision

CHEW tries to do something simple but powerful. We want to write "how the restaurant makes money" and "how the restaurant makes decisions" into a public set of on-chain rules.

More specifically:

- **Put profit rights on-chain.**
  All income, expenses, and distributable profit go through the `RestaurantAccounts` contract. Every year on December 31, the system takes a snapshot of all CHEW balances. On January 1, it sends mUSDT dividends to holders according to this snapshot. The rule is fixed in the contract, so everyone sees the same logic.

- **Put governance rights on-chain.**
  Big topics like choosing the location, renovation plan, or annual budget must go through the governance contracts (Governor + Timelock). Token holders create proposals and vote. If a proposal passes, it is executed after a delay, not instantly. Lighter topics, like menu updates or Top-N dishes, are handled by `MenuVoting`, so the community can join without touching the main treasury.

- **Align long-term incentives.**
  The team keeps 25% of CHEW, but these tokens are locked for 24 months. For two years, they cannot be sold and the team must stay in the same boat as investors. This makes it clearer that we are not here for a quick pump-and-dump, but for long-term performance of the restaurant.

4 System Overview

At a high level, CHEW is a "shareholder system on-chain" for a restaurant. All money and all major decisions must pass through a few fixed paths.

No matter if it is fundraising, daily revenue, dividend distribution, or governance voting, everything ends up going through six core smart contracts:

- **CHEWToken** – shows how many "shares" and how much voting power you have in the restaurant.
- **mUSDT** – a stablecoin we use on testnet, as a stand-in for real-world dollars.
- **Crowdsale** – the fundraising module, where investors pay mUSDT and receive CHEW.
- **RestaurantAccounts** – the treasury + accounting contract. It receives money, pays expenses, and calculates profit.
- **GovernanceProposal** – the main governance module for "big decisions", with a built-in time delay (Timelock).
- **MenuVoting** – a lighter voting system for menu-related choices like seasonal dishes or Top-N items.

4.1 How does money flow in the system?

We can think of the money flow in three main stages.

4.1.1 Fundraising stage

1. Investors send mUSDT to the **Crowdsale** contract.
2. The Crowdsale contract mints CHEW at a fixed price and sends the tokens to the investors.
3. When the fundraising ends (either we reach 300k mUSDT or the end date), the Crowdsale closes.
4. All raised mUSDT is moved into **RestaurantAccounts** as the initial capital for the restaurant.

4.1.2 Daily operations

1. Each day, the restaurant's real-world sales are converted into mUSDT and deposited into **RestaurantAccounts**. You can think of this as "moving the offline revenue into an on-chain record".
2. Operational expenses such as renovation costs, ingredients, rent, and salaries are also paid out from RestaurantAccounts. Every payment leaves an on-chain record.
3. At the end of each period (for example, monthly), we calculate "distributable profit" as revenue minus all costs. This profit stays in the treasury and is prepared for future dividends.

4.1.3 Annual dividend stage

1. On December 31 each year, **CHEWToken** takes a snapshot of all addresses and their CHEW balances.
2. On January 1 of the next year, **RestaurantAccounts** distributes mUSDT to all holders, in proportion to their balance in this snapshot.
3. If someone does not claim their dividend immediately, the money stays in the contract as a claimable balance. They can come back later and withdraw it at any time.

4.2 How are decisions made in the system?

4.2.1 Governance contracts

For big decisions, we use a governance flow based on **GovernanceProposal** and a **Timelock**.

Examples of such decisions include:

- Which location in Manhattan to choose,
- Whether to approve a larger renovation budget,
- How to set the annual operating budget.

The flow is:

1. Any address that holds enough CHEW can create a proposal in GovernanceProposal.
2. All token holders can vote during the voting period.
3. If the proposal passes, it does not execute immediately. Instead, it enters a Timelock delay (for example, 2 days). This gives the community time to review or react.
4. When the delay time is over, the governance contract automatically calls the target contract (usually **RestaurantAccounts**) and executes the action, such as "send X mUSDT from the treasury to pay for renovation".

4.2.2 Menu voting

For lighter topics, we use **MenuVoting**.

Typical examples:

- Which new dishes to add next quarter,
- Ranking Top-N dishes that the community wants to see on the menu.

These decisions do not move any money directly. They mainly collect community preferences and give the operating team a transparent reference.

This design increases participation, but keeps the core treasury safe. High-frequency menu voting happens in one layer, while sensitive fund movements and rule changes are always handled by the stricter governance layer.

5 Tokenomics

5.1 4.1 Total supply and initial allocation

CHEW is minted once at deployment. We create a fixed total supply, and after that there is no more minting. The smart contract does not have any function to mint extra tokens later. Tokens can only move between addresses; the total number can never increase out of thin air.

Within this fixed supply:

- **75%** goes to the community and outside investors. They buy CHEW with mUSDT in the Crowdsale contract. Our fundraising goal is **300,000 mUSDT**.
- The remaining **25%** is reserved for the team. This part does not unlock immediately. All of it goes into a **24-month lockup**:
  - During the lockup, these tokens cannot be transferred or sold.
  - On-chain they still count as real holdings, so they can join governance voting and receive dividends.
  - After 2 years, this chunk unlocks at once, and the team can move or sell it like any normal holder.

The idea is to make sure the team stays aligned with investors for the long term and cannot just sell everything in the first months.

5.2 4.2 Annual dividend rules

Once the restaurant starts normal operations, all income and expenses are recorded on-chain by the `RestaurantAccounts` contract.

At the end of each calendar year, we calculate the distributable profit for that year. This is the profit after rent, salaries, ingredients and other operating costs. We call this number `TotalDividends` (measured in mUSDT). It is the pool that will be shared by all CHEW holders.

We use a simple "record date + payout date" design:

- **Record date (Snapshot):** Every year on December 31, `CHEWToken` takes a snapshot of all addresses and their CHEW balances. It records exactly how many tokens each address holds on that day.
- **Payout date:** On January 1 of the next year, we distribute `TotalDividends` based on this snapshot.

For any address i, the dividend they should receive for that year is:

$$\text{payout}_i = \text{TotalDividends} \times \frac{\text{balance}_i(\text{SnapshotID})}{\sum_j \text{balance}_j(\text{SnapshotID})}.$$

Dividends do not go straight to the wallet. First, the contract records a claimable balance for each holder. A holder can call a claim function later to withdraw the mUSDT.

If someone forgets to claim in that year, it does not expire. The money stays in the contract under that address until they decide to claim it.

5.3 4.3 Secondary market trading and snapshot impact

After the Crowdsale ends and tokens are distributed, CHEW becomes a normal ERC20 token. It can be traded freely on secondary markets or transferred between wallets.

However, dividends only depend on the record date snapshot, not on short-term trading after that. We follow three simple rules:

- If you buy and hold before the record date, you can join that year's dividend.
- If you buy after the record date, you only get dividends starting from the next year.

- If you sell after the record date, it does not change the dividend already fixed for that year.

This means short-term trading is mostly about price movement. What really decides how much dividend you get is your share of CHEW on December 31.

5.4 4.4 Treasury and operating cash flow split

The way we handle money inside the restaurant is also written into the contract. `RestaurantAccounts` keeps track of two key balances on-chain:

- `operatingBalance` (operating account): used to pay rent, utilities, salaries and other day-to-day expenses.
- `reserveBalance` (reserve account): holds the real remaining profit. Later, this reserve is the source of `TotalDividends` for the year.

Inside the contract, there is a monthly function called `monthlySettlement()`. It looks at the revenue of the month and splits it between these two accounts based on a few parameters:

- `revenue`: total income of the month (stored in `receivingBalance`).
- `profitMargin`: target profit rate. For example, 70% means we aim for 70% to go to "profit + reserve" and 30% counted as variable costs.
- `fixedCosts`: the total fixed costs for the month, including rent, salaries, utilities, etc.

The logic is:

1. First, we conceptually split revenue into two parts:

$$\text{Variable cost part} = \text{revenue} \times (1 - \text{profitMargin}),$$

$$\text{Target profit part} = \text{revenue} \times \text{profitMargin}.$$

2. Then we add `fixedCosts` on top, and prioritize paying all bills from the operating account.
   - All amounts that must be paid immediately (variable costs + fixed costs) go into `operatingBalance`.
   - Only what is truly left over goes into `reserveBalance`.
3. If the revenue is too low to pay all bills, the contract will automatically move money from `reserveBalance` back into `operatingBalance` to cover the gap. Operations come first; dividends only happen if there is real surplus.

Example.

- Monthly revenue: 3,000 mUSDT.
- Target profit margin: 70% (so 30% treated as cost portion).
- Fixed costs: 2,000 mUSDT.

Then:

$$\begin{aligned}
\text{Cost portion} &= 3{,}000 \times 30\% = 900,\\
\text{Fixed costs} &= 2{,}000,\\
\text{Total needed in operating account} &= 900 + 2{,}000 = 2{,}900,\\
\text{Leftover for reserve} &= 3{,}000 - 2{,}900 = 100.
\end{aligned}$$

In this example, the operating account gets 2,900 (about 96.7% of cash), and the reserve only gets 100.

If revenue were even lower, the system might even pull from `reserveBalance` to keep `operatingBalance` high enough. This design makes sure the restaurant can survive and pay its basic bills first. Only the part that is truly extra profit will flow into the reserve and later become dividends for CHEW holders.

6 DAO System

6.1 GovernanceProposal

In CHEW, any decision that really changes the rules or moves a lot of money must go through the `GovernanceProposal` contract. You can think of it as an on-chain shareholder meeting.

Who can create a proposal? Any address that holds a certain amount of CHEW (and therefore has real voting power) can create a proposal in the contract. Typical proposals include:

- adjusting the target profit margin or dividend ratio of the restaurant;
- updating fixed cost parameters, such as rent or utilities;
- approving a large one-time payment for renovation or new equipment;
- using part of the reserve to support operations when the business is under pressure.

Each proposal must include a title, a short text description, and the target action (for example, which parameter to change, or how much money to send and to whom).

When a proposal is created, the system also records a snapshot block. For the whole voting process of this proposal, everyone's voting weight is calculated based on their CHEW balance at this snapshot block. This prevents people from buying a large amount of CHEW only during the vote and then dumping it right after, just to influence one single decision.

Voting phase During a fixed voting period (for example 7 days), all CHEW holders can vote *For*, *Against*, or *Abstain* on the proposal. The weight of each vote equals that address's voting power at the snapshot block. When the voting period ends, the contract counts all votes and computes the percentage of "For" votes. Only if "For" votes reach the threshold (for example at least 60% of all votes) is the proposal marked as passed.

Execution phase A passed proposal does not stop at "approved" status on a website. It goes into an on-chain execution step. The governance contract reads the proposal type and calls specific functions on the treasury and accounting module (`RestaurantAccounts`). In this way it can:

- update an operating parameter,
- send funds to a given supplier,
- move money between the operating account and the reserve account, etc.

During the whole lifecycle, the contract emits events such as `ProposalCreated`, `VoteCast`, and `ProposalExecuted`. This makes every step of governance traceable and easy to audit on-chain.

6.2 MenuVoting

Next to "serious governance", CHEW also has a lighter participation layer called `MenuVoting`. It is designed for community co-creation around the menu. `MenuVoting` never moves money from the treasury. It only answers one question:

> "Which dishes should we focus on this season?"

The `MenuVoting` process runs in seasons, usually one per quarter. At the start of each new season, the contract opens a new voting round and records a new snapshot block. All voting power in that season is based on CHEW balances at this snapshot, following the same "snapshot voting, no last-minute buying" idea.

In each season:

- Any user can submit candidate dishes, including the dish name and a short description.
- The operations team can filter out candidates that are clearly not suitable, for example because they break cost structure or food safety rules.

- After the candidate list is confirmed, the voting starts. Addresses that hold CHEW can vote for multiple dishes. Every time they vote, that dish receives the voter's full weight for this season's snapshot.

When the voting period ends, the contract sorts all dishes by total votes and selects the top group (for example the Top 20). This set of dishes becomes the "on-chain menu result" for the current quarter. The contract emits events (such as `SeasonFinalized`) with the final ranking, so anyone can verify the outcome. The restaurant team then uses this list as a main reference for real-world menu updates and marketing.

Importantly, `MenuVoting` by itself does not trigger any fund transfers or budget changes. If a menu decision requires extra spending, it still has to go back to the `GovernanceProposal` system and pass a formal vote.

By separating serious governance from light participation, CHEW achieves two goals at the same time:

- The treasury and core parameters are protected by strict voting and execution rules.
- Ordinary token holders can still join the brand and menu building process with a low entry barrier.

Together, `GovernanceProposal` and `MenuVoting` form the DAO system of CHEW: money and rules follow clear on-chain paths, and everyday community interactions also leave a verifiable trace on the blockchain.

7 Conclusion

The CHEW project tries to answer a simple but often ignored question: Can a real-world restaurant put both "making money" and "making decisions" on top of a public, verifiable set of on-chain rules?

To explore this, we designed and implemented an on-chain shareholder system centered around the CHEW token. It includes a fundraising contract, a treasury and accounting module, an annual dividend mechanism, and a DAO layer with governance and menu voting. In this paper we explain how these pieces work together.

From the workflow side, our assignment went through two main stages:

1) Choosing the scenario and abstracting the problem. As a team we first compared several possible DAO ideas, and finally chose the "restaurant" setting because it is close to daily life and the problems are easy to understand. Then we turned the real-world pain points — opaque books, unclear dividend rules, black-box decisions, and weak community participation — into protocol requirements. We saw that we needed: (1) a standard way to raise funds and represent shareholders, (2) a clear on-chain path from business revenue to distributable profit, and (3) a governance framework where votes actually lead to execution.

2) Splitting the system into modules and building it together. After the requirements were clear, we split the system into several modules: `Crowdsale`, `CHEWToken`, `RestaurantAccounts`, the dividend rules, `GovernanceProposal`, and `MenuVoting`. Different teammates focused on different parts: smart contract coding and testing, fund flow and state machine design, tokenomics and documentation, etc. Through many rounds of alignment we made sure that "what the code does" and "what the whitepaper says" are based on the same assumptions and parameters.

During this process, we learned several important lessons:

Translating real business into contracts forces precision. Things that can be fuzzy offline must become concrete variables, thresholds, and state changes on-chain: Should we protect `operatingBalance` first or `reserveBalance`? How do we define the record date and payout date? How long is the lockup? Being "forced to write it down" made us understand the restaurant business model more deeply.

The key in governance is not voting itself, but how votes execute. With the design of snapshot + voting period + delayed execution, a passed proposal is automatically executed by the governance contract, which calls controlled functions in `RestaurantAccounts`. This updates parameters or moves funds without needing an off-chain decision. In this way, one CHEW token represents not only a vote on a website, but a piece of governance power that can be replayed and audited on-chain.

There is a trade-off between decentralization and real-world operations. If we try to put "everything on-chain for voting", the system becomes very hard to run. In the end we introduced role-based permissions, a two-year team lockup, proposal thresholds, and execution delays. These choices are a compromise between safety, efficiency, and practical operations, and are closer to what a real restaurant might actually use.

Designing a protocol is also an exercise in teamwork. If assumptions are not made clear at the start, everyone imagines a different "restaurant" and a different "set of rules". By repeatedly asking "what exact problem are we solving" and "what should an investor see on the interface", we learned how to break a vague idea into several parallel modules, and how to keep the interfaces and events consistent across the team.

Overall, the most important takeaway from this assignment is that "building a DAO" is not only a technical exercise in blockchain. It is also a way to rethink how real-world business logic works and how people cooperate around shared rules and incentives.